

SLIP – 22

Section I: Data Structures [15 Marks]

Q1) Read the data from the "employee.txt" file and sort on age using Quick sort and write the sorted data to another file 'sortedemponage.txt'. Assumed file format (one record per line): Name Age e.g. Raj 30

Answer:

```
#include <stdio.h>

struct Emp
{
    char name[50];
    int age;
};

void quicksort(struct Emp a[], int low, int high)
{
    int i = low, j = high;    int pivot =
a[(low + high) / 2].age;    struct Emp
temp;

    while(i <= j)
    {
        while(a[i].age < pivot)
i++;        while(a[j].age >
pivot)        j--;

        if(i <= j)
        {
            temp = a[i];
a[i] = a[j];
a[j] = temp;
i++;        j--;
        }
    }

    if(low < j)
quicksort(a, low, j);    if(i
< high)
        quicksort(a, i, high);
}

int main()
{
```

```

    FILE *fp, *out;
    struct Emp e[100];    int n
    = 0, i;

    fp = fopen("employee.txt", "r");
    if(fp == NULL)
    {
        printf("File not found");
    return 0;
    }

    while(n < 100 && fscanf(fp, "%s %d", e[n].name, &e[n].age) == 2)
n++;
    fclose(fp);

    if(n == 0)
    {
        printf("No records");
    return 0;
    }

    quicksort(e, 0, n - 1);

    out = fopen("sortedemponage.txt", "w");
    if(out == NULL)
    {
        printf("Cannot create output file");
    return 0;
    }

    printf("Sorted data:\n");
    for(i = 0; i < n; i++)
    {
        fprintf(out, "%s %d\n", e[i].name, e[i].age);
        printf("%s %d\n", e[i].name, e[i].age);
    }
    fclose(out);

    return 0;
}

```

Section II: Database Management Systems [15 Marks]

Q 2) Consider the following database(Primary keys are underlined)

Employee(eno,ename,designation,salary)

Department(dno,dname,location)

The relationship is as follows: Employee-Department: One to Many.

Constraint: employee salary greater than 5000

Q 2(A)

A.Create the above database in PostGreSQL and insert sufficient records. [5 marks]

Answer:

```
CREATE TABLE Department(  
    dno INT PRIMARY KEY,      dname  
    VARCHAR(50),      location  
    VARCHAR(50)  
);
```

```
CREATE TABLE Employee(  
    eno INT PRIMARY KEY,  
    ename VARCHAR(50),      designation VARCHAR(50),  
    salary NUMERIC(10,2) CHECK(salary > 5000),  
    dno INT REFERENCES Department(dno)  
);
```

```
INSERT INTO Department VALUES  
(1, 'HR', 'Nanded'),  
(2, 'Sales', 'Pune'), (3, 'IT', 'Mumbai');
```

```
INSERT INTO Employee VALUES  
(101, 'Sagar', 'Clerk', 20000, 1),  
(102, 'Amit', 'Manager', 50000, 2),  
(103, 'Anita', 'Developer', 40000, 2);
```

Q 2(B)

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. Change designation of employee “Sagar” to Manager.

Answer:

```
UPDATE Employee  
SET designation = 'Manager' WHERE  
ename = 'Sagar';
```

ii. Display the minimum salary of an employee.

Answer:

```
SELECT MIN(salary) FROM Employee;
```

iii. List the details of all the departments located at Nanded city.

Answer:

```
SELECT * FROM Department  
WHERE location = 'Nanded';
```

iv. Display the details of employees whose names starts with A.

Q 2(C)

C. Write a query to list the department name-wise total salary. [4 marks]

Answer:

```
SELECT D.dname, SUM(E.salary) AS total_salary  
FROM Department D  
JOIN Employee E ON D.dno = E.dno  
GROUP BY D.dname;
```

Q 3) Viva [5 marks]